

FSDI 115

User Experience and Responsive Design

CLASS 3

BUILD

BRING YOUR DESIGN TO LIFE

We will transform our designs into a responsive website using HTML, CSS and responsive design techniques.



COURSE OBJECTIVE

Learn how to build a responsive website from your design, applying HTML, CSS and responsive design principles.

Plan
Code
Create
Learn ♡

0:00 – 0:15

Welcome & Recap



- Quick recap of Class 2
- Today's agenda
- Q&A



0:15 – 0:45

From Design to Code



- How designs become websites
- Intro to HTML
- Structure and essential elements



0:45 – 1:10

Styling with CSS



- What is CSS
- Selectors and properties
- Layouts and simple styling techniques



1:10 – 1:20

Break



Take a break
and recharge!



1:20 – 2:00

Responsive Design



- What is responsive design
- Media queries
- Mobile-first approach
- Best practices



2:00 – 2:45

Build the Main Layout



- Implement the structure
- Apply styles
- Make it responsive
- Work on your project (guided practice)



2:45 – 3:00

Wrap Up & Next Steps



- Key takeaways
- Assignment 3 overview
- Questions



Design
Build
Grow ♡



SAME PROJECT. BIGGER SKILLS.
TURNING IDEAS INTO REAL EXPERIENCES.



You got this! ♡

FROM Design to Code

Turn ideas into real websites

Same idea.
Different tools.
Real results!



WHAT WILL YOU LEARN?

How to take a visual design and transform it into a working website using HTML and CSS.

Design
Inspires.
Code
Brings it to Life. ♥

1 START WITH A DESIGN

We begin with a visual design (a wireframe or mockup) that shows what the website should look like.



The design shows the structure, content and layout.

2 ANALYZE THE DESIGN

We identify the main elements of the design and plan how to build them with code.



Break the design into smaller parts. This makes it easier to code.

3 WRITE THE HTML (STRUCTURE)

We use HTML to create the structure of the page.



HTML is like the skeleton of the website. It defines what content goes where.

4 ADD THE CSS (STYLE)

We use CSS to make the page look good: colors, fonts, spacing, and layout.



CSS brings the design to life. It controls the visual style and makes it look like the design.

5 SEE THE RESULT IN THE BROWSER

We open the HTML file in a browser and check that the website looks like the design. Then we make adjustments if needed.



Small steps, big results! You just turned a design into a real website.



DESIGN GIVES YOU THE VISION.
CODE MAKES IT REAL.

Same design. New skills. More possibilities. ♥



DESIGN. CODE. CREATE.
REAL WEBSITES.



FROM MOCKUP TO HTML

Every visual element in your design has an **HTML** equivalent.

1 LOOK at your mockup

Identify each visual element.



2 CHOOSE the HTML element

Use the right tag for each element.



3 BUILD the structure

Organize elements with semantic HTML.

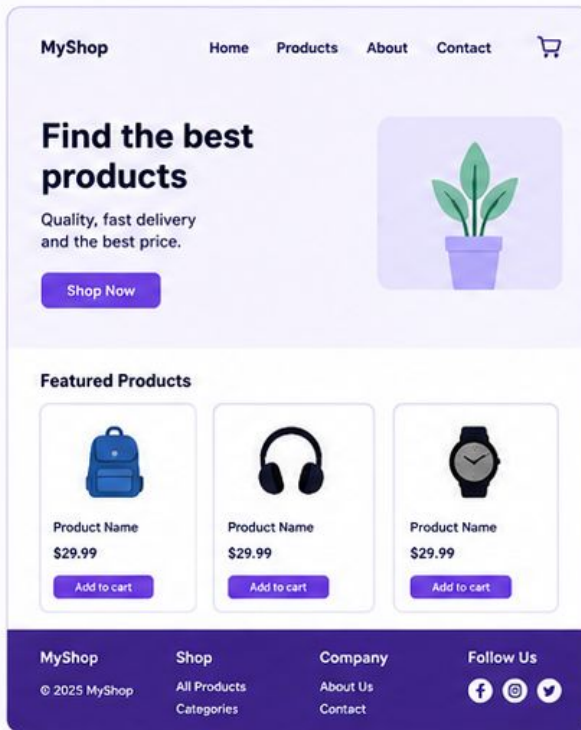


COMMON ELEMENTS MAPPING

MOCKUP ELEMENT	HTML ELEMENT	EXAMPLE
Heading / Title	<code><h1> - <h6></code>	<code><h1>Welcome</h1></code>
Paragraph / Text	<code><p></code>	<code><p>This is a description.</p></code>
Image	<code></code>	<code></code>
Button	<code><button></code>	<code><button>Get Started</button></code>
Input Field	<code><input></code>	<code><input type="text" placeholder="Email"></code>
Navigation Menu	<code><nav></code>	<code><nav> ... </nav></code>
Card / Container	<code><div></code>	<code><div class="card"> ... </div></code>
List	<code> / / </code>	<code> Item </code>
Link	<code><a></code>	<code>Learn more</code>
Section	<code><section></code>	<code><section> ... </section></code>
Footer	<code><footer></code>	<code><footer> ... </footer></code>

EXAMPLE: DESIGN TO CODE

MOCKUP (FIGMA)



HTML STRUCTURE

```
<header>
  <nav> ... </nav>
</header>

<section class="hero">
  <div class="hero-content">
    <h1>Find the best products</h1>
    <p>Quality, fast delivery
      and the best price.</p>
    <button>Shop Now</button>
  </div>
  
</section>

<section class="products">
  <h2>Featured Products</h2>
  <div class="product-grid">
    <article class="product-card"> ... </article>
    <article class="product-card"> ... </article>
    <article class="product-card"> ... </article>
  </div>
</section>

<footer>
  <div class="footer-content"> ... </div>
</footer>
```

BEST PRACTICES



Use semantic HTML

Choose tags that describe the meaning of the content.



Think in sections

Break your page into header, main, sections and footer.



Mobile first

Start with the content structure, then style it for different screens.



Clean & organized code

Indent, close your tags and use meaningful class names.



Accessibility matters

Use alt for images, proper headings and contrast.



Remember: Good HTML structure = Easy to style + Easy to maintain + Accessible for everyone



Build the structure today,
style it beautifully with CSS!

STYLING with CSS

GIVE YOUR WEBSITE A VISUAL STYLE

Same structure.
A whole new look! ♥



WHAT IS CSS?

CSS (Cascading Style Sheets) is used to control the visual style of a web page: colors, fonts, spacing, layout and more.

Style it.
Make it yours.
Bring your design to life! ♥

1 SELECT ELEMENTS

We target HTML elements using selectors.

```
h1 {  
  color: #2563eb;  
}
```

Selectors tell the browser which elements to style (e.g., h1, p, .class, #id).

2 CHANGE COLORS

We can set colors for text, backgrounds and more.

```
body {  
  background-color: #f9fafb;  
  color: #333;  
}
```

Colors help create visual hierarchy and make the design look clean and consistent.

3 ADJUST FONTS

We can control the font family, size, weight and alignment.

```
h1 {  
  font-family: 'Segoe UI',  
    sans-serif;  
  font-size: 32px;  
  font-weight: bold;  
  text-align: center;  
}
```

Typography improves readability and gives personality to your site.

4 ADD SPACING

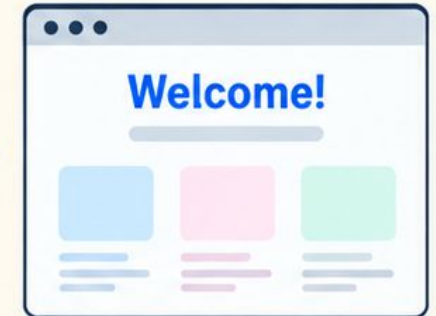
We use margin and padding to create space between elements.

```
.card {  
  padding: 20px;  
  margin: 16px;  
}
```

Spacing makes the layout easier to read and less cluttered.

5 SEE THE RESULT

We refresh the browser to see our styled website.



Small CSS changes make a big difference!



SAME HTML.
MORE STYLE.
BIGGER IMPACT.

Good design is not just how it looks,
but how it feels. ♥



PRACTICE. EXPERIMENT.
MAKE IT YOURS.



Responsive Design

ONE WEBSITE. EVERY DEVICE. A GREAT EXPERIENCE.

Same content.
Different screens.
Happy users! ❤️



RESPONSIVE DESIGN

makes your website look and work great on any device: desktop, tablet or mobile.

1

START WITH A FLEXIBLE LAYOUT

Use relative units and modern CSS layout techniques like Flexbox or Grid.

```
.container {  
  display: flex;  
  flex-wrap: wrap;  
  gap: 1rem;  
}
```



Flexible layouts adapt automatically to different screen sizes.

2

USE FLEXIBLE IMAGES AND MEDIA

Make images resize within their containers.

```
img {  
  max-width: 100%;  
  height: auto;  
  display: block;  
}
```



Images should never overflow the screen.

3

USE MEDIA QUERIES

Apply different styles for different screen sizes (breakpoints).

```
@media (max-width: 768px) {  
  .menu {  
    flex-direction: column;  
  }  
}
```

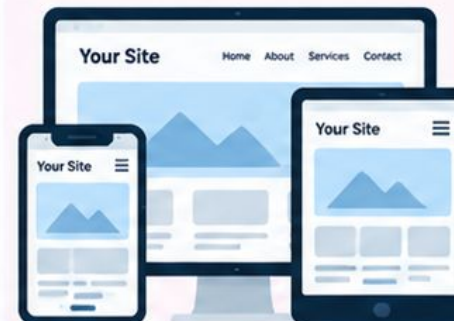


Adjust the layout, font sizes or hide/show elements based on the screen size.

4

TEST ON DIFFERENT DEVICES

Check your website on desktop, tablet and mobile (or use browser dev tools).



Make sure the experience is consistent and easy to use on all devices.

5

KEEP IT SIMPLE

Use a mobile-first approach and focus on the essential content.



Clear navigation



Readable text



Proper spacing



Touch-friendly buttons (on mobile)



Fast loading

Good design works everywhere! ❤️



RESPONSIVE DESIGN
= HAPPIER USERS
= BIGGER IMPACT

Design once. Reach everyone.
That's the power of responsive design. ❤️



SAME CONTENT.
MORE OPPORTUNITIES.



RESPONSIVE DESIGN

Build websites that look great on every screen

1 THE BIG QUESTION

Should a website look the same on a phone and on a 27" screen?

Everyone will say...

NO.



2 WHAT IS RESPONSIVE DESIGN?

Responsive Design allows a website to adapt to different screen sizes.



3 HOW DOES CSS KNOW WHEN TO CHANGE?

With a **Media Query**.

4 WHAT IS A MEDIA QUERY?

It's simply a condition.
Very similar to an if statement.

IN PROGRAMMING (IF)

```
if (screen < 768) {  
  // do this  
}
```

IN CSS (MEDIA QUERY)

```
@media (max-width: 768px) {  
  /* do this */  
}
```

TRANSLATION

@media (max-width: 768px) means:
If the screen is smaller than 768 pixels... do this.



5 EXAMPLE 1: CARDS LAYOUT

DESKTOP

Three columns.



MOBILE

One column.



HOW?

Very easy!

DESKTOP

```
.cards {  
  display: flex;  
}
```

↑
Items in a row

MOBILE

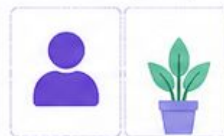
```
@media (max-width: 768px) {  
  .cards {  
    flex-direction: column;  
  }  
}
```

↑
Items in a column

6 EXAMPLE 2: TEXT & IMAGE

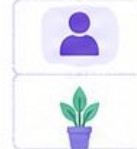
DESKTOP

Text and image side by side.



MOBILE

One below the other.



HOW?

You only change one property:

```
flex-direction: column;
```

7 THE 3 THINGS THAT WILL CHANGE IN MOBILE

Nothing more.

1 NAVBAR

DESKTOP

Home Products About Contact

MOBILE

Home
Products
About
Contact

2 HERO SECTION

DESKTOP

Text Image

MOBILE

Text
Image

3 CARDS SECTION

DESKTOP



MOBILE



8 WHAT YOU NEED TO REMEMBER



FLEXBOX

Organizes elements automatically.

You only need:

- display: flex;
- justify-content;
- align-items;
- gap;



MEDIA QUERY

Changes the layout based on screen size.

You only need:

```
@media (max-width: 768px) {  
  /* rules */  
}
```

THE ANALOGY



Flexbox is like organizing the furniture in a living room.

You decide if they go in a row, in a column, centered or spaced.



Media Queries are like having folding furniture.

When the room is big, you can place them side by side. When it's small, you reorganize them so everything is comfortable.

THE TRICK (AND THE CONFIDENCE BOOST)



You do NOT need to memorize CSS.

You only need to understand two ideas:

- Flexbox decides how elements are arranged.
- Media Queries decide when the layout should change.

IN ONE SENTENCE

Flexbox controls **HOW** things are arranged.
Media Queries control **WHEN** the layout changes.



BUILD THE MAIN LAYOUT

Organize. Structure. Build.

CREATE THE STRUCTURE OF YOUR WEBSITE

Use what you've learned to build the main layout of your project.



GOAL

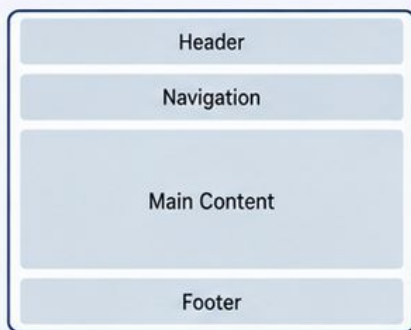
Create the main layout of your website using semantic HTML elements and CSS layout techniques.

Good structure builds great websites! ❤️



1 IDENTIFY THE SECTIONS

Divide your page into the main sections (e.g., header, navigation, main content, footer).



Think about the structure before coding. Keep it simple and clear.

2 WRITE THE HTML STRUCTURE

Use semantic HTML elements to create the skeleton of your page.

```
<header>...</header>
<nav>...</nav>
<main>...</main>
<section>...</section>
<footer>...</footer>
```



Use semantic elements. They help with accessibility and make your code easier to understand.

3 APPLY CSS LAYOUT TECHNIQUES

Use CSS (Flexbox or Grid) to organize the sections on the page.

```
body {
  display: flex;
  flex-direction: column;
  min-height: 100vh;
}

main {
  flex: 1;
}
```



Flexbox and Grid help you create clean and flexible layouts.

4 ADD BASIC STYLES

Apply basic styles to visualize the layout (e.g., background colors, spacing, borders).

```
header {
  background-color: #2563eb;
  padding: 20px;
}

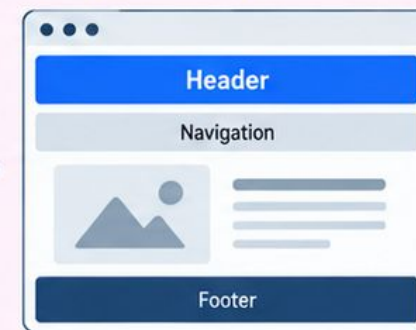
main {
  padding: 20px;
  background-color: #f9fafb;
}
```



Use colors and spacing to see the structure of your layout.

5 CHECK THE RESULT

Open your page in the browser and verify that all sections are displayed correctly.



Make sure the layout looks clean, organized and is ready for the next step.



A SOLID LAYOUT
IS THE FOUNDATION
OF A GREAT WEBSITE.

Structure today. Amazing websites tomorrow. ❤️



SAME STEPS.
BIGGER POSSIBILITIES.

WHAT IS FLEXBOX?

Flexbox is a CSS layout system that automatically arranges elements inside a container.

In other words...

Flexbox helps us organize content without calculating positions manually.



REAL LIFE EXAMPLE

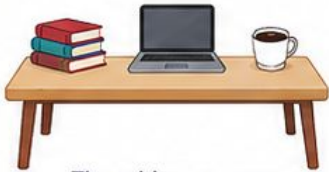
Imagine a table.

WITHOUT FLEXBOX



You have to move each object.

WITH FLEXBOX



The table arranges them for you.



Flexbox = Smart table

You decide how items should be arranged, and Flexbox does the work!

THE SECRET OF FLEXBOX

There are always two things.

1 The Container (Parent)

```
<div class="cards">
  ...
</div>
```

2 The Elements (Children)

```
<div class="cards">
  <article></article>
  <article></article>
  <article></article>
</div>
```

TO ACTIVATE FLEXBOX

You only need one line.

```
.cards {
  display: flex;
}
```



That's it!

Now the children are automatically arranged.

THE 4 PROPERTIES YOU NEED TO KNOW

1 display: flex;

Activates Flexbox.

```
display: flex;
```



2 justify-content

Moves elements horizontally.

```
justify-content: center;
```

Result:



```
justify-content: space-between;
```

Result:



```
justify-content: space-around;
```

Result:



3 align-items

Aligns items vertically.

```
align-items: center;
```

Result:



Everything is centered vertically.

4 gap

Adds space between items.

```
gap: 20px;
```

Result:



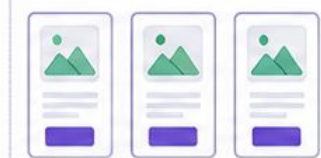
Creates consistent space between items.

EXAMPLE: CARD SECTION

A Card Section looks like this:

```
.cards {
  display: flex;
  justify-content: center;
  gap: 30px;
}
```

Automatically you get:



No position calculations. Flexbox does it for you! 🦄



REMEMBER

You control the container. Flexbox arranges the items.

NOW: RESPONSIVE DESIGN

Here's a question...



Should a website look the same on a phone and on a 27" screen?

Everyone will say:

No.

Perfect! 😊

Responsive Design

allows a website to adapt to different screen sizes.



Different screens = Different layouts.



Key Idea

Flexbox decides HOW items are arranged. Media Queries decide WHEN the layout should change.



THE ONLY 2 IDEAS YOU NEED



FLEXBOX

Decides how elements are arranged.



MEDIA QUERIES

Decides when the layout should change.



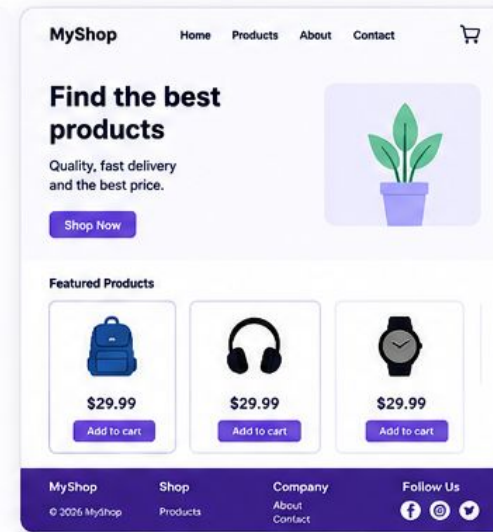
MASTER THESE TWO IDEAS AND YOU WILL CREATE RESPONSIVE WEBSITES!



LAB 1: FROM MOCKUP TO HTML

Build this responsive landing page using **HTML** & **CSS**

🕒 Total Time: 60 Minutes 🎯 Goal: Recreate the webpage exactly like the design



WHAT WE WILL USE



HTML



CSS



Flexbox



Media Queries

STEP-BY-STEP PLAN

1 SETUP 5 min



Create your project and files

- Create index.html
- Create styles.css
- Create images folder

```
myshop/  
├── index.html  
├── styles.css  
└── images/
```

★ TIP: Start mobile first!
It helps you design better layouts.

2 BUILD THE HTML STRUCTURE 15 min



Add the HTML structure

- Header (nav + logo + menu)
- Hero section
- Products section (3 cards)
- Footer

```
<header>...</header>  
<section class="hero">  
  ...  
</section>  
<section class="products">  
  ...  
</section>  
<footer>...</footer>
```

💡 TIP: Use semantic HTML5 tags.

3 STYLE THE LAYOUT 20 min



Use CSS & Flexbox

- Style the header
- Style the hero section
- Create product cards
- Style the footer

```
.navbar {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
}  
  
.cards {  
  display: flex;  
  gap: 20px;  
}
```

★ TIP: Flexbox is your best friend!

4 ADD IMAGES & ICONS 10 min



Place images and icons

- Add hero image (plant)
- Add product images
- Add icons (cart, social media)



💡 TIP: Use images with transparent background.

5 MAKE IT RESPONSIVE 15 min



Add Media Queries

- Stack elements on mobile
- Adjust font sizes
- Make cards responsive

```
@media (max-width: 768px) {  
  .navbar ul {  
    display: none;  
  }  
  .cards {  
    flex-direction: column;  
    align-items: center;  
  }  
}
```

★ TIP: Test on different screen sizes!

6 REVIEW & FINISH 10 min



Check your final result

- Review spacing & alignment
- Check colors & fonts
- Test all screen sizes



💡 TIP: Small details make a big difference!



WHAT YOU WILL ACHIEVE

- ✓ A complete responsive landing page
- ✓ Clean and organized HTML
- ✓ Modern CSS with Flexbox
- ✓ Mobile, tablet and desktop views



SUCCESS CRITERIA
Your page should look Exactly like the mockup on all screen sizes.



REMEMBER
Good code today,
Great developer tomorrow!



TODAY'S CODE STRUCTURE

Organized • Simple • Easy to navigate

We will build our responsive website using these files and folders.



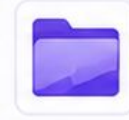
index.html

The main HTML file. This is the entry point of our website.



styles.css

The CSS file that contains all the styles for our website.



images/

Folder that contains all the images used in our website.



plant.png

Image used in the hero section (plant).



watch.png

Image used in the products section (watch).



headphones.png

Image used in the products section (headphones).



backpack.png

Image used in the products section (backpack).



WHY THIS STRUCTURE?

- ✓ Keeps our project clean and organized.
- ✓ Easy to find and update files.
- ✓ Good practice for real-world projects.



TIP

Always keep your images in the **images/** folder and link them correctly in your HTML.

</> EXAMPLE TO LINK AN IMAGE

```

```



GOOD HABIT

A well-organized project saves time and makes coding easier!



MINI CHALLENGE

⌚ TIME: 5 MINUTES



WITHOUT MY HELP...

Make these changes to your landing page on your own!



GOAL

Customize your page and practice what you just learned!

MAKE THESE 4 CHANGES

1 CHANGE BUTTON COLOR



Pick a new color for all buttons.

BEFORE

Shop Now



AFTER

Shop Now

2 CHANGE HERO IMAGE



Use a different image in the hero section.

BEFORE



AFTER



3 CHANGE CARD BACKGROUND



Choose a new background color for the product cards.

BEFORE



Backpack
\$29.99



AFTER



Backpack
\$29.99

4 CHANGE FOOTER COLOR



Pick a new background color for the footer.

BEFORE



AFTER



CHALLENGE RULES

- ✓ Do it without asking for help.
- ✓ Explore and try different options.
- ✓ Use what you learned today.
- ✓ Have fun and be creative!



TIPS

- ✓ Use your CSS file.
- ✓ Try different colors and images.
- ✓ Refresh the browser to see your changes.
- ✓ If something breaks, don't worry — keep trying!



YOU DID IT!

You've just customized your page and practiced your skills like a real developer!

Small changes, big impact! 🚀



EXPERIMENT • EXPLORE • IMPROVE



PRACTICE MAKES PROGRESS!



KEEP CODING, KEEP GROWING!



PRACTICE EXERCISE

Local Coffee Shop

From idea to a real, responsive website — in just 1 hour!

CLIENT REQUEST

"We are a local coffee shop and we need a simple website to let people know who we are, what we offer and how to contact us. It should look modern, work on mobile devices and be easy to navigate."
— The Coffee Shop Team

YOUR GOAL

Follow the steps to design and build a simple, responsive website for the coffee shop.



Good ideas brew great websites! ♥

1 ANALYZE THE REQUEST

🕒 5 min

Review the client request and identify what they need.

Functionalities required:

- ✓ Home page with a short introduction
- ✓ Menu (coffee, drinks, food)
- ✓ About us section
- ✓ Contact information
- ✓ Responsive design (mobile-friendly)



Deliverable:
List of requirements (in your own words).

2 DEFINE PEOPLE AND FLOW

🕒 5 min

Identify who will use the website and how they will navigate it.

Create:



User personas
(e.g., customer, new visitor)



User flow
(main steps to achieve the goal, e.g., find menu, get location)



Deliverable:
Personas and a simple user flow diagram.

3 CREATE A SKETCH

🕒 5 min

Draw a simple sketch of the main pages.

Include:

- ✓ Home
- ✓ Menu
- ✓ About
- ✓ Contact



Deliverable:
Hand-drawn or digital sketch of the layout.

4 BUILD THE WIREFRAME (HTML)

🕒 15 min

Create the basic structure in HTML (no colors yet).



Focus on:

- Use semantic HTML elements (header, nav, main, section, footer)
- Add placeholder content (titles, text, images).

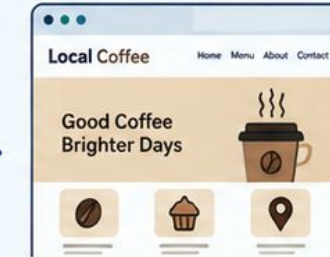


Deliverable:
A working HTML structure in the browser.

5 ADD THE MOCKUP (HTML + CSS)

🕒 20 min

Apply visual styles to match the coffee shop brand.



Focus on:

- Add colors, fonts and spacing
- Include images or icons
- Make it look clean and appealing



Deliverable:
A styled version of the website (mockup) in the browser.

6 ADD INTERACTIVITY & RESPONSIVENESS

🕒 10 min

Make it responsive and add simple interactions.



Focus on:

- Use CSS media queries (for mobile view)
- Add simple interactions (e.g., hover effect on buttons)
- Test on different screen sizes



Deliverable:
A responsive and interactive website.



FINAL RESULT

A simple, responsive website for a local coffee shop, built from analysis to prototype — in 1 hour!



TIPS FOR SUCCESS

- ✓ Keep it simple
- ✓ Use placeholder text and images
- ✓ Focus on the main sections
- ✓ Test on desktop and mobile



Small project.
Big learning!



From Ideas to Real Websites

THINK → DESIGN → BUILD

Same process.
Bigger possibilities. ♥



**TURN YOUR IDEAS
INTO REAL EXPERIENCES**
Plan. Design. Build. Learn. Grow.

Good
websites
start with
great ideas! ♥



THINK

UNDERSTAND AND PLAN

Ask
Explore
Define

Start with the user, define the purpose and plan your website.

1



Understand the User

Identify the target audience and their needs.

2



Define the Purpose

What is the goal of your website?
What information will it include?

3



Plan the Content

List the main sections and organize the information.



Outcome: A clear idea of what you will build and who it is for.



DESIGN

CREATE THE VISUAL SOLUTION

Sketch
Organize
Visualize

Transform your ideas into a visual design that communicates and inspires.

1



Create Wireframes

Plan the layout and structure of each page (low-fidelity).

2



Design the Look & Feel

Create the visual design (mockups) with colors, typography and images.

3



Consider Responsive Design

Make sure the design works well on different devices (desktop, tablet, mobile).



Outcome: A visual design ready to be built.



BUILD

BRING YOUR DESIGN TO LIFE

Code
Style
Test

Use HTML and CSS to turn your design into a responsive website.

1



Create the HTML Structure

Use semantic HTML elements to build your pages.

2



Style with CSS

Apply colors, fonts, spacing and layout techniques (Flexbox or Grid).

3



Make It Responsive

Test and adjust the layout for different screen sizes.



Outcome: A fully functional, responsive website.



SAME IDEAS.
NEW SKILLS.
REAL WEBSITES.

Think it. Design it. Build it. ♥



PRACTICE TODAY.
CREATE TOMORROW.

Wrap Up

REFLECT • CELEBRATE • LOOK AHEAD

Same learning.
New skills.
Real progress. 



TODAY'S GOAL

Finish strong, recognize what you accomplished, and get ready for what's next.

You did it!
Keep going! 

1

REVIEW

WHAT WE COVERED



From Design to Code

Turned ideas into a real structure using HTML.



Styling with CSS

Added colors, fonts, spacing and layout.



Responsive Design

Made the website look great on different screen sizes.



Build the Main Layout

Organized the content using semantic elements and layout techniques (Flexbox/Grid).

 You now have the skills to turn designs into real, responsive websites!

2

KEY TAKEAWAYS

WHAT YOU LEARNED



HTML Structure

Use semantic elements to organize content.



CSS Styling

Apply colors, typography, spacing and layout.



Responsive Techniques

Use media queries to adapt the layout to different devices.



From Design to Code

Transform a visual design into a working website.



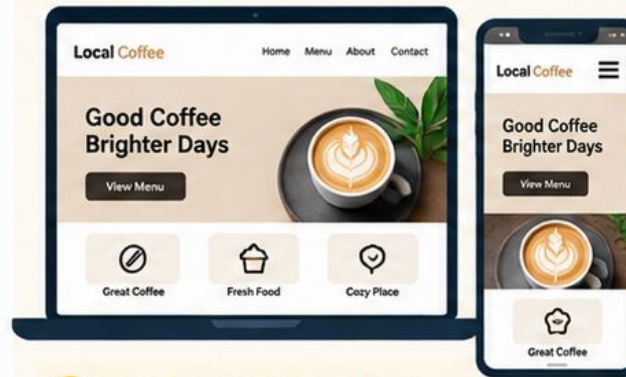
Iterate and Improve

Small changes make a big difference.

3

WHAT YOU BUILT

A REAL PROJECT



- ✓ A clean and structured HTML layout
- ✓ A styled and visually appealing design
- ✓ A responsive website (desktop, tablet and mobile)
- ✓ A functional navigation menu
- ✓ A real example you can be proud of!

4

NEXT STEPS

KEEP LEARNING



Practice more

Try new ideas and personalize your project.



Explore new topics

Animations, forms, or JavaScript features.



Get feedback

Share your work and learn from others.



Build your portfolio

Add this project to showcase your skills.



Keep going!

You're one step closer to creating amazing web experiences.



TODAY YOU TURNED
KNOWLEDGE INTO ACTION.
THAT'S REAL PROGRESS.

Build what you imagine. 



SAME SKILLS TODAY.
BRIGHTER OPPORTUNITIES TOMORROW.



ASSIGNMENT 3 (AFTER CLASS)

Responsive Landing Page Project



Due before Session 4
at 11:59 p.m.

This assignment is
worth 20% of your grade.



YOUR GOAL

Transform the approved mockup
into a fully responsive landing page
using HTML & CSS.

It should look great on all devices!

WHAT YOU NEED TO BUILD

Create a complete responsive landing
page based on the provided mockup.



- ✓ Desktop (1200px and up)
- ✓ Tablet (768px – 1199px)
- ✓ Mobile (up to 767px)

★ Make it look as close as possible
to the mockup!

WHAT TO INCLUDE



Header / Navigation
Logo, menu links, responsive
navigation (hamburger on mobile).



Hero Section
Title, subtitle, call to action button,
hero image.



Products / Features Section
Cards or items using Flexbox.



Why Choose Us / Benefits
Icons + short descriptions.



Contact or Call to Action Section
Encourage the user to take action.



Footer
Links, social icons, and copyright.

TECHNOLOGIES & CONCEPTS TO USE



HTML5
(Structure)



CSS3
(Styling)



Flexbox
(Layout)



Responsive
Design



Media Queries
(Breakpoints)



Images
& Assets



Use what you learned today:
semantic HTML, Flexbox, and Media Queries
to make your page responsive.

REQUIREMENTS

- ✓ Use semantic HTML.
- ✓ Use Flexbox for layouts.
- ✓ Use Media Queries for responsive design.
- ✓ Your page must work on desktop, tablet and mobile.
- ✓ Code should be clean, organized and well commented.
- ✓ All images must be in the **images/** folder.

WHAT TO SUBMIT



Submit a ZIP file with:

- images/
- index.html
- styles.css



Test your project on different
screen sizes before submitting!



SUCCESS TIPS



Plan your structure
before coding.



Mobile first mindset:
start small, then grow.



Keep your design
consistent.



Test often on different
devices & sizes.



YOU CAN DO IT!

One step at a time. Keep building,
keep learning, keep improving! 🚀



YOU ARE BUILDING **REAL SOLUTIONS**



DESIGN → CODE → TEST



PRACTICE MAKES PROGRESS



SMALL STEPS TODAY,
BIG RESULTS TOMORROW



KEEP BUILDING!
YOU'RE BECOMING A DEVELOPER!







Thank you!!.

User Experience and Responsive Design